

On the Power of Tree-Walking Automata

Frank Neven^{1,*} and Thomas Schwentick²

¹ Limburgs Universitair Centrum

² Johannes Gutenberg-Universität Mainz, Institut für Informatik

Abstract. Tree-walking automata (TWAs) recently received new attention in the fields of formal languages and databases. Towards a better understanding of their expressiveness, we characterize them in terms of transitive closure logic formulas in normal form. It is conjectured by Engelfriet and Hooeboom that TWAs cannot define all regular tree languages, or equivalently, all of monadic second-order logic. We prove this conjecture for a restricted, but powerful, class of TWAs. In particular, we show that 1-bounded TWAs, that is TWAs that are only allowed to traverse every edge of the input tree at most once in every direction, cannot define all regular languages. We then extend this result to a class of TWAs that can simulate first-order logic (FO) and is capable of expressing properties not definable in FO extended with regular path expressions; the latter logic being a valid abstraction of current query languages for XML and semi-structured data.

Keywords: automata, logic, regular tree languages

1 Introduction

Regular tree languages can be defined by means of many equivalent formalisms, for instance: (non)deterministic bottom-up and nondeterministic top-down tree automata, alternating tree automata, two-way tree automata, homomorphic images of local tree languages, and monadic second-order logic [13,20]. However, it is not known whether there exists a natural inherently sequential model for recognizing the regular tree languages. Of course, by definition, they are recognized by bottom-up finite tree automata, but these automata are essentially parallel rather than sequential: the control of the automata is at several nodes of the input tree simultaneously, rather than at just one. With this aim in mind, Engelfriet, together with his co-workers Bloem, Hooeboom, and van Best, initiated a research program [5,8,10] studying (extensions of) the tree-walking automata (TWAs) originally introduced by Aho and Ullman [2]. The finite control of a tree-walking automaton is always at one node of the input tree. Based on the label of that node and its child number (which is i if it is the i th child of its parent), the automaton changes state and steps to one of the neighboring nodes (parent or child). Without the test on the child number such automata cannot even search the tree in a systematic way, such as by a pre-order traversal as

* Research Assistant of the Fund for Scientific Research, Flanders.

is shown by Kamimura and Slutzki [14]. However, also with the child test, it is conjectured that these automata cannot express all regular tree languages [8,10]. In this paper, we study the expressiveness of tree-walking automata by characterizing them in terms of transitive closure logic formulas in normal form and prove the above mentioned conjecture for a restricted, but powerful, class of tree-walking automata.

Apart from the above purely theoretical motivation, recently, new interest in tree-walking automata emerged from the field of database theory. Indeed, one of the major research topics at the moment is the design and study of query languages for the manipulation of XML documents or electronic documents in general [1,16]. Such documents are usually modeled by ordered labeled trees or graphs, depending on the application at hand. In this research, tree-walking automata are used for various purposes and appeared in various forms. Milo, Suciu, and Vianu [15], for instance, used a transducer model based on tree-walking automata as a formal model for an XML transformer encompassing most current XML transformation languages. Brüggeman-Kleinn, Hermann, and Wood [6], proposed to use *caterpillar expressions* as a pattern language for XML transformation languages. Interestingly, caterpillar expressions relate to tree-walking automata like regular expressions relate to string automata: they are just a different, though a lot more user friendly, representation of the same thing. Furthermore, they conjectured their formalisms to be less expressive than the regular tree languages. Another, more direct, occurrence of tree-walking automata is embodied in the actual XML transformation language XSLT [7] proposed by the World Wide Web consortium (W3C) and currently being implemented by IBM. In formal language theoretic terms, this query language can be best described as a *tree-walking tree transducer* [4]. Hence, results on the expressiveness of tree-walking automata could give insight in the expressiveness of actual XML transformation languages.

We start by characterizing the expressiveness of (deterministic and nondeterministic) tree-walking automata in terms of (*deterministic and non-deterministic*) *transitive closure logic* (DTC and TC) formulas in normal form. That is, formulas of the form $[(D)TC(\varphi)](\varepsilon, \varepsilon)$, where φ is an FO formula containing predicates $\text{depth}_m(x)$ defining x as a vertex whose depth is a multiple of m ; and where ε refers to the root of the tree under consideration. Our result thus implies that any lower bound on (D)TC formulas in normal form is also a lower bound for (non)deterministic tree-walking automata. Unfortunately, proving lower bounds for the latter logic does not seem much easier than the original problem as Ehrenfeucht games for DTC and TC are quite involved. Therefore, we use a direct approach for a restricted, but expressive, class of tree-walking automata in the hope that these techniques will provide insight for the general case.

We first show that 1-bounded tree-walking automata, that is tree-walking automata that are only allowed to traverse every edge of the input tree at most once in every direction, cannot define all regular languages. In particular, we obtain that they can not evaluate tree-structured Majority circuits where the gates have fan-in greater than 2. Next, we generalize this result to a rather

powerful class of tree-walking automata, called r -restricted. These automata are rather expressive as they can define all of first-order logic (FO) and are capable of expressing some tree languages not definable in FO extended with regular path expressions. The latter logic is an abstraction of current query languages for semi-structured data and XML [1,16], and, for instance, cannot define the set of trees representing Boolean circuits evaluating to true [17] which can easily be defined by r -restricted tree-walking automata (cf. Example 1).

We conclude by mentioning some related work. Bargury and Makowsky [3] proved an equivalence between transitive closure logic and two-way multihead automata operating on grids. Their simulation of automata involves nesting of TC operators. Potthoff [18] showed that the same normal form of TC we use, suffices to define all regular *string* languages, the opposite direction being trivial in the string case. Recently, Engelfriet and Hoozeboom [9] showed that tree-walking automata with pebbles correspond exactly to TC. Hence, when allowing pebbles one can simulate nested TC operators. Fülöp and Maneth [11] showed that the domains of partial attributed tree transducers correspond to the tree-walking automata in universal acceptance mode.

This article is structured as follows. In Section 2, we define tree-walking automata. In Section 3, we prove the logical characterization of tree-walking automata in terms of transitive closure logic, and in Section 4 we proof the Engelfriet and Hoozeboom conjecture for two restrictions of tree-walking automata.

2 Preliminaries

Trees. A *tree domain* τ over $\mathbb{N}$ is a subset of $\mathbb{N}^*$, such that if $v \cdot i \in \tau$, where $v \in \mathbb{N}^*$ and $i \in \mathbb{N}$, then $v \in \tau$. Here, $\mathbb{N}$ denotes the set of natural numbers. If $i > 1$ then also $v \cdot (i - 1) \in \tau$. The empty sequence, denoted by ε , represents the root. We call the elements of τ *vertices*. A vertex w is a *child* of a vertex v (and v the *parent* of w) if $vi = w$, for some i . A Σ -tree is a pair $t = (\text{dom}(t), \text{lab}_t)$, where $\text{dom}(t)$ is a tree domain over $\mathbb{N}$, and lab_t is a function from $\text{dom}(t)$ to Σ . The *arity* of a tree is the maximum number of children of its vertices. We only consider trees with a fixed arity. The depth of a vertex u , denoted by $\text{depth}(u)$ is the length of u (interpreted as a string over $\mathbb{N}^*$). The *distance* between two vertices u and v , denoted by $d(u, v)$ is defined as the length of the path between u and v where we assume that $d(u, u) = 0$.

A Σ -tree t can be naturally viewed as a finite structure over the binary relation symbols E and $<$, and the unary relation symbols $(O_\sigma)_{\sigma \in \Sigma}$. E is the edge relation and equals the set of pairs $(v, v \cdot i)$ for every $v, v \cdot i \in \text{dom}(t)$. The relation $<$ specifies the ordering of the children of a node, and equals the set of pairs $(v \cdot i, v \cdot j)$, where $i < j$ and $v \cdot j \in \text{dom}(t)$. For each σ , O_σ is the set of nodes that are labeled with a σ .

Tree-Walking Automata. Tree-walking automata (TWAs) can be seen as the simplest analogon of two-way string automata. A TWA starts its computation in an initial state at the root of the input tree. In each step, it moves to a neighbor

vertex of the current vertex (or stays at the current vertex) and enters a state. The direction of movement and the new state depend only on the current state, the symbol at the current vertex, the child number of the current vertex, i.e., the relative position of the current vertex in the ordered list of the children of its parent, and the number of children of the current vertex.

More formally, a (k -ary) TWA is a tuple $(S, \Sigma, \delta, s_0, F)$, where S is the set of states, Σ is an alphabet (the set of possible vertex labels), $s_0 \in S$ is the initial state and $F \subseteq S$ is the set of accepting states. The only part where a TWA is formally different from a standard string automaton is the transition function δ . The transition function δ of a TWA is the union of the functions δ^i and $\delta^{\text{root},i}$, where $i \in \{0, \dots, k\}$. For a deterministic TWA,

- $\delta^{\text{root},i}$ is a function from $S \times \Sigma$ to $\{\text{stay}, \downarrow_1, \dots, \downarrow_i\} \times S$, and
- for each $i \in \{0, \dots, k\}$, δ^i is a function from $\{1, \dots, k\} \times S \times \Sigma$ to $\{\uparrow, \text{stay}, \downarrow_1, \dots, \downarrow_i\} \times S$.

For a nondeterministic TWA the ranges of these functions are the respective power sets. If several TWAs are around we write δ_M to denote the transition function of TWA M and the like.

A configuration $c = [v, s]$ of a TWA M on a tree t consists of a vertex v of t and a state s of M . The immediate successor configuration c' of a configuration $[v, s]$ is defined as follows.

- If $v = \epsilon$, v has i children and carries the symbol σ then
 - $c' = [\epsilon, s']$, if $\delta^{\text{root},i}(s, \sigma) = (\text{stay}, s')$, and
 - $c' = [j, s']$, if $\delta^{\text{root},i}(s, \sigma) = (\downarrow_j, s')$.
- If $v = wj$, for some $j \leq k$, v has i children and carries the symbol σ then
 - $c' = [w, s']$, if $\delta^i(j, s, \sigma) = (\uparrow, s')$,
 - $c' = [v, s']$, if $\delta^i(j, s, \sigma) = (\text{stay}, s')$, and
 - $c' = [vj', s']$, if $\delta^i(j, s, \sigma) = (\downarrow_{j'}, s')$.

We write $c \Rightarrow_{M,t} c'$ to express that c' is the immediate successor configuration of c in the computation of M on t . Following standard convention we write $c \Rightarrow_{M,t}^j c'$ ($c \Rightarrow_{M,t}^* c'$) to express that c' is the j -th (some) successor configuration of c in the computation of M on t . If M and/or t are clear from the context we may omit them. A tree t is *accepted* by M if there is an $s \in F$ such that $[\epsilon, s_0] \Rightarrow_{M,t}^* [\epsilon, s]$.

Example 1. We illustrate the above definition by means of an example. In particular we define a deterministic tree-walking automaton that accepts all tree-structured Boolean circuits of fan-in 2 that evaluate to true. A similar construction can be given for each fixed bound on the fan-in. For convenience, we only consider circuits of the right format. That is, all inner nodes and the root have exactly two children and are labeled with AND or OR. Further, all leaves are labeled by 0 or 1. These circuits are assigned a truth value in the usual way. Define M as the tuple $(S, \Sigma, \delta, \text{eval}, F)$ with $S = \{\text{eval}, 0, 1, \text{left-child-0}, \text{left-child-1}\}$, $\Sigma = \{\text{AND}, \text{OR}, 0, 1\}$, and $F = \{1\}$. The transition function δ is defined as follows:

- M starts by walking to the first leaf in the tree: for each $\sigma \in \{\text{AND}, \text{OR}\}$ and $i \in \{1, 2\}$,

$$\begin{aligned}\delta^{\text{root}, 2}(\text{eval}, \sigma) &= (\downarrow_1, \text{eval}); \text{ and} \\ \delta^2(i, \text{eval}, \sigma) &= (\downarrow_1, \text{eval}).\end{aligned}$$

- When reaching a zero (one) leaf, which is a left child, M moves up with this information:

$$\begin{aligned}\delta^0(1, \text{eval}, 0) &= (\uparrow, \text{left-child-0}); \\ \delta^0(1, \text{eval}, 1) &= (\uparrow, \text{left-child-1});\end{aligned}$$

- If M enters an inner vertex from a left child it is always in one of the two states left-child-0 or left-child-1. If the current vertex is an AND vertex and the state is left-child-0 then the current vertex will evaluate to 0 no matter the right subtree. An analogous statement holds for OR-gates and the state left-child-1. The information passed to the parent vertex depends on whether the current vertex is itself a left or a right child. If the outcome of the left child is not sufficient to determine the value of the current vertex M has to enter its right child. Formally, for each $i \in \{1, 2\}$:

$$\begin{aligned}\delta^2(1, \text{left-child-0}, \text{AND}) &= (\uparrow, \text{left-child-0}); \\ \delta^2(2, \text{left-child-0}, \text{AND}) &= (\uparrow, 0); \\ \delta^2(1, \text{left-child-1}, \text{OR}) &= (\uparrow, \text{left-child-1}); \\ \delta^2(2, \text{left-child-1}, \text{OR}) &= (\uparrow, 1); \\ \delta^2(i, \text{left-child-1}, \text{AND}) &= (\downarrow_2, \text{eval}); \text{ and} \\ \delta^2(i, \text{left-child-0}, \text{OR}) &= (\downarrow_2, \text{eval}).\end{aligned}$$

- If M enters an inner vertex from a right child it is always in one of the states 0 or 1. By what we have said before, this state indicates the value of the subtree at the current vertex. Hence, it only has to be passed to its parent vertex. Consequently, for each $i \in \{0, 1\}$ and $\sigma \in \{\text{AND}, \text{OR}\}$:

$$\begin{aligned}\delta^2(1, 0, \sigma) &= (\uparrow, \text{left-child-0}); \\ \delta^2(1, 1, \sigma) &= (\uparrow, \text{left-child-1}); \text{ and} \\ \delta^2(2, i, \sigma) &= (\uparrow, i).\end{aligned}$$

- It remains to handle the case of leaves that are right children of their parent. They simply have to pass their value to the parent. For each $i \in \{0, 1\}$:

$$\delta^0(2, \text{eval}, i) = (\uparrow, i).$$

It will be convenient to subsume the overall effect of a TWA on a subtree of a tree in a so-called behaviour function. Intuitively, if f is the behaviour function of a subtree t then $f(s) = s'$ if and only if in the computation of M which starts at the root v of t in state s the parent of v is entered in state s' . Obviously, the behaviour function of a subtree t depends on the child number of its root in the full tree.

We define behaviour functions more formally. Let M be a k -ary deterministic TWA and let t be a (at most) k -ary tree. For $i \leq k$, let $t(i)$ denote the tree which consists of a root which has the root of t as the i -th child and $i-1$ other children which are leaves. The behaviour function $f_{M,t,i}$ of M on t as an i -th child maps states of M to states of M . It is defined as follows. If s is a state of M then $f_{M,t,i}(s) = s'$, if there is a j such that $(i, s) \Rightarrow_{M,t(i)}^j (\epsilon, s')$ and j is minimal with this property.

Note that $f_{M,t,i}$ does not depend on the labels of the vertices outside of t . Furthermore it does not depend either on the actual embedding of t in a larger tree as long as the root of t is an i -th child.

For non-deterministic TWAs behaviour functions are defined analogously but with sets of states as function values.

Next, we turn to the definition of some classes of restricted TWAs.

- We call a TWA *1-bounded*, if, for all trees t , it traverses each edge of t at most once in each direction.
- We call a TWA M *r -restricted* if the following holds. For each pair u, v of vertices of a tree t such that $d(u, v) > r$ the computation of M on t does not contain 4 configurations $[u, s_1], [v, s_2], [u, s_3], [v, s_4]$ in the given order. Intuitively, this means that each path of length more than r is traversed at most once in each direction.

Clearly, each 1-bounded TWA is also r -restricted for every $r \geq 1$. Engelfriet and Hooeboom showed that the latter automata can define all of first-order logic (FO) [8]. Moreover, r -restricted TWAs can even define some tree languages not definable in FO extended with regular path expressions [17]. In Section 4, we show that r -restricted TWAs cannot define the set of all regular tree languages thereby giving an answer to the conjecture of Engelfriet and Hooeboom for a powerful class of TWAs.

Let, for $m > 0$, depth_m be a unary relation symbol. In the following we will consider trees which have additionally the predicate depth_m , for some m . In all trees, depth_m will contain all vertices the depth of which is a multiple of m . For a vertex u of a tree t , its r -sphere $S_r^t(u)$ is the set $\{v \mid d(u, v) \leq r\}$. For a tuple of vertices $\bar{u}$, define $S_r^t(\bar{u})$ as $\bigcup_{u \in \bar{u}} S_r^t(u)$. We define its r -neighborhood $N_r^t(\bar{u})$ as the structure t extended with the constants $\bar{u}$ restricted to the set $S_r^t(\bar{u})$.

3 A Logical Characterization of Tree-Walking Automata

We characterize tree-walking automata by *transitive closure logic formulas* of the form $\text{TC}[\varphi(x, y)](\varepsilon, \varepsilon)$, where φ is an FO formula, which may make use of the predicate depth_m , for some m . We say that such a formula is in *normal form*. Further,

$$t \models \text{TC}[\varphi(x, y)](\varepsilon, \varepsilon)$$

iff the pair $(\varepsilon, \varepsilon)$ is in the transitive closure of the relation $\{(u, v) \mid t \models \varphi[u, v]\}$. We use *deterministic transitive closure logic formulas* (DTC) in an analogously

defined normal form to capture deterministic tree-walking automata. In particular,

$$t \models \text{DTC}[\varphi(x, y)](\varepsilon, \varepsilon)$$

iff the pair $(\varepsilon, \varepsilon)$ is in the transitive closure of the relation $\{(u, v) \mid t \models \varphi[u, v] \wedge (\forall z)(\varphi[u, z] \rightarrow z = v)\}$. The latter expresses that we disregard vertices u that have multiple φ -successors.

Theorem 2. *Nondeterministic tree-walking automata accept precisely the tree languages definable by TC formulas in normal form. Deterministic tree-walking automata accept precisely the tree languages definable by DTC formulas in normal form.*

Proof. (sketch) The simulation of TWA's by (D)TC formulas in normal form is an easy extension of a proof of Potthoff [18] who characterized two-way *string* automata by means of TC formulas in normal form. We omit the details.

For the other direction, consider the formula $\text{TC}[\varphi(x, y)](\varepsilon, \varepsilon)$. By Gaifman's Theorem (see, e.g., [12]), there exists an r such that φ is equivalent to a Boolean combination of sentences χ and r -local formulas $\xi(x, y)$. Here, a formula $\xi(x, y)$ is r -local if for every tree t with vertices u and v , whether $t \models \xi[u, v]$ only depends on the isomorphism type of $N_r^t(u, v)$. As the rank of trees is fixed, there are only finitely many possible isomorphism types. Engelfriet and Hogeboom [8] showed that TWA's can evaluate FO formulas over trees. A straightforward adaptation of their proof shows that TWA's can also evaluate FO formulas with modulo depth predicates (details omitted). Hence, we can assume that M can evaluate the sentences χ at the beginning of the computation. Further, if $d(u, v) \leq 2r$ then M can check whether $t \models \varphi[u, v]$ by inspecting $N_{3r}^t(u)$, otherwise by first inspecting $N_r^t(u)$ and afterwards $N_r^t(v)$. Suppose the automaton arrives at a vertex u (with $u = \varepsilon$ as the first case). First, the automaton nondeterministically chooses whether it will go to a vertex v of distance $\leq 2r$ or to a vertex v of distance $> 2r$. In the first case, it inspects $N_{3r}^t(u)$ and chooses a v such that $t \models \varphi[u, v]$ if this is possible. Otherwise it first computes the type of $N_r^t(u)$, moves to a vertex v of distance $> 2r$ and checks that the type of $N_r^t(v)$ implies that $t \models \varphi[u, v]$. Finally, if v is the root, the automaton accepts. Otherwise, it proceeds in the same manner. Clearly, the automaton will eventually accept if $t \models \text{TC}[\varphi(x, y)](\varepsilon, \varepsilon)$.

The simulation of DTC formulas by deterministic TWAs is more intricate. Consider the formula $\text{DTC}[\varphi(x, y)](\varepsilon, \varepsilon)$. We construct a deterministic TWA over k -ary trees accepting exactly the k -ary trees the above formula defines. As in the previous case, the automaton first evaluates all sentences χ in the Gaifman normal form of φ . Let m be the maximum number of vertices occurring in an r -neighborhood of a tree. That is, $m := \max\{|S_r^t(u)| \mid t \text{ a tree, } u \in \text{dom}(t)\}$. For each isomorphism type τ of r -neighborhoods with one distinguished vertex, the automaton additionally computes the number of occurrences of τ in t up to $m + 2$. Now, given a u , to find a v such that $\varphi(u, v)$ holds, the automaton proceeds as follows. Let Y_0 be the set of all vertices w in $N_{2r}^t(u)$ such that

$N_r^t(u, w) \models \varphi(x, y)$. By inspecting the $3r$ -neighborhood of u , M can compute $|Y_0|$.

Next, it computes the type of $N_r^t(u)$ and the set T of all types τ of r -neighborhoods for which the following holds: if $N_r^t(w)$ is of type τ and $N_r^t(w)$ and $N_r^t(u)$ are disjoint then $N_r^t(u, w) \models \varphi(x, y)$. The latter is a fixed finite computation which can be encoded into the transition function. We call vertices of a type from T *good* vertices (w.r.t. the current u) and denote the set of good vertices in t by Y_1 . Note that M can deduce $|Y_1|$ (up to $m + 2$) from the precomputed information.

The set of good vertices in $N_{2r}^t(u)$ is denoted by Y_2 . By inspecting the $3r$ -neighborhood of u again, M computes $|Y_2|$ and the relative positions of all vertices in Y_2 w.r.t. u .

Let $Y = Y_0 \cup (Y_1 - Y_2)$. Y is the set of vertices w of t such that $N_r^t(u, w) \models \varphi(x, y)$. M can compute $|Y|$ without further moving (note that $|Y_2| \leq m$). If $|Y|$ is different from 1 then M can immediately reject. Let us assume in the following that $|Y| = 1$.

If the unique element v of Y is from Y_0 , M can directly go to v . If, on the other hand, $Y_0 = Y_2 = \emptyset$ then M can move to the unique $v \in Y_1$ via a DFS traversal of the tree.

The only complicated case is when $Y_0 = \emptyset$ and $|Y_1 - Y_2| = 1$ but $Y_2 \neq \emptyset$. The complication arises from the following possibility. M has to traverse the tree to find the correct unique good w outside $N_{2r}^t(u)$. As it does not know in which part of the tree w is located its way to w might lead back to u . In that case we have to make sure that it does not confuse w with a vertex from Y_2 .

We can assume w.l.o.g. that the root of the tree is not in $N_{3r}^t(u)$ because otherwise M can easily distinguish the vertices of Y_2 from the desired vertex. Now, M proceeds as follows. It starts a DFS walk through the tree starting from u and first inspecting the first subtree of u . Whenever it encounters a new vertex z it starts a subcomputation which inspects the $3r$ -neighborhood of z to find out whether there is a good vertex w in $N_{2r}^t(z)$. If such w is found then M computes the set $Z(w)$ of vertices $u' \in N_{2r}^t(w)$ for which $N_{3r}^t(u') \cong N_{3r}^t(u)$. If $Z(w) = \emptyset$ or all vertices in $Z(w)$ are behind z in the DFS order then w is the desired vertex v and M goes there. Otherwise it proceeds in its DFS walk. When the DFS walk finishes at the root (without finding the target vertex) then M walks back (reverse DFS) until it reaches u again (easily recognized by the isomorphism type of its $3r$ -neighbourhood). Then it starts a reverse DFS walk from u (going upwards first) analogously to the first DFS walk.

We have to show that M always finds the correct target vertex. First, we show that M never moves to a vertex in Y_2 . Let $w \in Y_2$. If M reaches w during the inspection of the neighborhood of a vertex z before its DFS walk arrives at the root then $u \in Z(w)$ and M recognizes that u is in the DFS order before z . Hence it does not take w as v . The analogous statement is true if z is found in the reverse DFS walk after coming back to u .

Finally, we show that M indeed reaches a target vertex v from Y_1 (which then is the correct one). Assume wlog that v is encountered as a vertex w relative

to a vertex z in the DFS walk behind u . If $Z(v) = \emptyset$ then v is easily identified. Assume $Z(v) \neq \emptyset$. As v is not in $N_{2r}^t(u)$ all vertices in $Z(v)$ are behind u in the DFS order. Let u' be the first vertex from $Z(v)$ in the DFS order. Then v is found not later than at the time u' is assumed as z in the DFS walk.

4 Weakness of Tree-Walking Automata

Let k be fixed and let T_k be the set of all k -ary trees the leaves of which are labeled with 0 or 1. For each vertex v of a tree $v \in T_k$ we inductively define a value 0 or 1 as follows. If v is a leaf then its value (0 or 1) is determined by its label (0 or 1). If v has i children then v has the value 1 if and only if at least $\frac{i}{2}$ of its children have the value 1. Intuitively, T_k is the set of all tree-structured Majority circuits. Let T_k^1 (T_k^0) denote the set of all trees $t \in T_k$ for which the root gets the value 1 (0). We call a vertex v of a tree $t \in T_k$ a *1-vertex* (*0-vertex*) if it gets the value 1 (0). Analogously, we call the subtree rooted at a 1-vertex (0-vertex) a *1-subtree* (*0-subtree*). The next lemma follows immediately from the inductive definition of the set T^k .

Lemma 3. *For each k , the set T_k^1 is a regular set of trees.*

We have seen in Example 1 that there is a deterministic TWA which recognizes T_2^1 (note that $T_2^1 \cup T_2^0$ can be seen as the set of trees representing Boolean circuits consisting of only OR-gates). This was due to the fact that the automaton, after evaluating a right subtree of a vertex v , could conclude the value of the corresponding left subtree of v from the label of v and the fact that it had to enter the right subtree. For $k > 2$, things are more complicated. In fact, we conjecture the following.

Conjecture 4. For $k > 2$, T_k^1 can not be recognized by a (det. or nondet.) TWA.

In this section we prove this conjecture for a restricted type of TWAs, 1-bounded TWAs. The proof can be easily generalized to the sets T_k^1 , for each $k > 3$. Furthermore, it can be extended to show that, for each r there is a related regular set of trees which can not be recognized by any r -restricted TWA.

Before we state and prove the result we introduce some important concepts for that proof and show a purely combinatorial result. For any $d \geq 1$ a *critical tree* of depth d is a full ternary tree t of depth d with the following properties: (i) $t \in T_3^1$; (ii) each 1-vertex of t has exactly two children which are 1-vertices; and (iii) each 0-vertex has only 0-vertices as children. In particular, there are no 1-leaves in 0-subtrees of t . Intuitively, a critical tree of depth d is a tree of depth d from T_3 which contains a full binary subtree of depth d which only has 1-leaves and all other leaves are 0-leaves. In particular, a critical tree of depth d has 2^d 1-leaves. A *numbering* N of a critical tree t of depth d is an injective mapping of the 1-leaves of t into the set $\{0, \dots, 2^d - 1\}$. All critical trees of a fixed size d are defined on the same tree domain τ_d . A mapping M which maps each leaf of τ_d to a subset of $\{0, \dots, 2^d - 1\}$ with at most m elements is called an *m-labeling*. We say that an *m-labeling* M of τ_d is *compatible* with a numbering

N of a critical tree t of depth d if, for each 1-leaf v of t , $N(v) \in M(v)$. We state the following lemma without proof.

Lemma 5. *For each $m > 0$ there is a $d > 0$ such that, for each m -labeling M of τ_d there is a critical tree of depth d for which there is no numbering that is compatible with M .*

We are now ready to prove the main result of this paper.

Theorem 6. (a) *There is no 1-bounded (deterministic or non-deterministic) TWA which recognizes T_3^1 .*

(b) *For each $r > 0$, there is a regular tree language that can not be recognized by an r -restricted (deterministic or non-deterministic) TWA.*

Proof. (sketch) The main task is to prove (a). Statement (b) will follow by an easy generalization of that proof. Towards a contradiction assume there exists a non-deterministic TWA M' which recognizes T_3^1 . The proof consists of 3 main steps:

- We transform M' into a TWA M which accepts exactly the same trees as M' but always rejects when it visits a 0-leaf. From this property we can conclude that if M accepts a certain tree t then it also accepts every tree which results from t by replacing 0-subtrees with 1-subtrees.
- We show that there are two trees $t, t' \in T_3^1$ with the same tree domain such that M has accepting computations for these trees which enter some vertex v in the same state s but with different “histories”. Here, the history consists of the information for which vertices w on the path from v to the root the computation visits the (only) 1-sibling of w before it visits v . This part of the proof makes use of Lemma 5.
- Finally, we show that these accepting computations can be combined into one accepting computation on a tree from T_3^0 .

Before we describe the construction of M , we introduce some notation. Let, for $i \in \{1, \dots, 3\}$, $F_{M',i}^1$ ($F_{M',i}^0$) denote the set $\{f_{M',t,i} \mid t \in T_3^1\}$ ($\{f_{M',t,i} \mid t \in T_3^0\}$) of behaviour functions that M' can have on 1-subtrees (0-subtrees) that have child number i . A straightforward argument shows that M' can never recognize the set T_3^1 when $F_{M',i}^1 \cap F_{M',i}^0 \neq \emptyset$ for some i . Therefore, we can assume that $F_{M',i}^1$ and $F_{M',i}^0$ are disjoint for all i .

We turn to the construction of M . Intuitively, the idea is as follows. Whenever M' can go from a vertex v to vi then M can either do the same or, to prevent visiting a 0-leaf, it can guess that vi is the root of a 0-subtree. In the latter case instead of going down to vi it picks a behaviour function $f \in F_{M',i}^0$ and enters a new state at v according to f . Formally, for each $j \leq k$, $s \in S$ and $\sigma \in \Sigma - \{0\}$,

$$\delta_M^i(j, s, \sigma) = \delta_{M'}^i(j, s, \sigma) \cup \bigcup_{\substack{(\downarrow_{j'}, s') \in \delta_{M'}^i(j, s, \sigma) \\ f \in F_{M',j'}^0}} (\text{stay}, f(s')).$$

For each $j \leq k$ and $s \in S$, we define $\delta_M^i(j, s, 0) = (\text{stay}, \perp)$ where $\perp$ is a state from which no transition is possible.

We have to show that M accepts exactly all trees in T_3^1 . Let therefore t be from T_3^1 , hence, by assumption, t is accepted by M' .

Let $C' = c'_0, c'_1, \dots, c'_n$ be an accepting computation of M' on t . We show that there is also an accepting computation $C = c_0, \dots, c_m$ of M on t . We construct C by suitably modifying C' . Let $v = wj$ be a 0-leaf of t , for some vertex w and some j . By the prerequisite on M' , v is at most visited once in C' . If v is not visited at all, we do not need to modify C' with respect to v . If v is visited then there are successive configurations $[w, s_i], [v, s_{i+1}], [w, s_{i+2}]$ in C' . Here, we assume w.l.o.g. that M' moves in each step. In C we replace these 3 configurations by $[w, s_i], [w, s_{i+2}]$. This reflects a legal transition of M as it corresponds to the computation of M' on the 0-subtree which consists of a single 0-leaf. We end up with a legal accepting computation C on t .

For the opposite direction, let $C = c_0, \dots, c_m$ be an accepting computation of M on a tree $t \in T_3$. We construct a tree t' by replacing some of the 0-leaves of t by 0-trees and an accepting computation C' of M' on t' by modifying C accordingly. By the construction of t' it will follow that $t \in T_3^1$ if and only if $t' \in T_3^1$. Hence, as M' accepts t' it also accepts t .

More formally, let $[v, s], [v, s']$ be two successive configurations from C such that it does not hold $[v, s] \Rightarrow_{M', t} [v, s']$. Hence, this subcomputation is possible only by the new transitions introduced in M . By definition, there must be $j \leq k$, $s'' \in S$, $f \in F_{M', j}^0$ and a 0-tree t_0 such that $[v, s] \Rightarrow_{M', t} [vj, s'']$ and $f_{M', t_0, j}(s'') = s'$. From this we can conclude that there exist configurations $c(v, 1), \dots, c(v, l)$ such that $[v, s], [vj, s''], c(v, l), \dots, c(v, l), [v, s']$ is a legal subcomputation of M' on the tree, denoted by $t(v)$, in which the subtree rooted at vj is replaced by the 0-subtree t_0 . If $t \in T_3^0$ then also $t(v) \in T_3^0$ as we only replaced a (0- or 1-) subtree by a 0-subtree. By inductively applying this argument we arrive at a tree t' and an accepting computation C' on t' . Hence, by assumption, $t' \in T_3^1$ and therefore $t \in T_3^1$.

It should be noted that the latter construction relies on the assumption that M' traverses each edge at most once. Otherwise, it could be the case that the configuration C visits v two times but the extension to C' makes use of two different 0-subtrees rooted at v .

Let now t be a critical tree of depth d and let $C = c_0, \dots, c_n$ be an accepting computation of M on t . If C does not visit all 1-leaves then we can easily construct a tree $t' \in T_3^0$ which is accepted by M . Hence, we assume that C visits each 1-leaf of t exactly once. Let v be such a 1-leaf of t and let $c_j = [v, s]$ be the configuration of C which visits it. Let, for $i \in \{1, \dots, d\}$, v_i denote the vertex of depth i on the path from the root of t to v . As v is a 1-leaf, each v_i is a 1-vertex. Therefore, as t is critical, each vertex v_i has exactly one sibling v'_i which is a 1-vertex. For each i , v'_i is visited in exactly one of the subcomputations $c_0, \dots, c_{j-1}$ and $c_{j+1}, \dots, c_m$. We define, for each 1-leaf v its *history string* $z = h_{t, C}(v) = z_1 \dots z_d$ by setting $z_i = 1$ if and only if v'_i is visited in $c_0, \dots, c_{j-1}$. These history strings have a couple of nice properties which are straightforward to prove given the assumptions on M .

- The binary number $b_{t,C}(v)$ represented by $z = h_{t,C}(v)$ (where z_d is the least significant bit) coincides with the number of 1-leaves that are visited in C before v . This follows from the restriction that the automaton can visit each subtree at most once. Hence, when a 1-vertex v at level i is entered, then all 2^{d-i} 1-leaves in the subtree of v have to be visited before the subtree is left. In this way, a 1 at the i -th bit of the history string corresponds to 2^{d-i} 1-leaves that have been already visited.
- Consequently, the function $b_{t,C}$ defines a numbering of t . In particular, each 0-1-string occurs exactly once as a history string of a 1-leaf v .

We claim that, there exists a d , two critical trees t, t' of depth d , two accepting computations C, C' of M on t, t' , respectively, and a leaf v of τ_d such that (i) v is a 1-leaf of t and t' ; (ii) C and C' visit v in the same state s ; (iii) and, $h_{C,t}(v) \neq h_{C',t'}(v)$.

Towards a contradiction assume that this claim is false. Let m be the number of states of M and let d be a number as given by Lemma 5. Let t and t' be two critical trees of depth d , let v be a common 1-leaf of t and t' and let C and C' be accepting computations of t and t' , respectively, which visit v in the same state s . The assumption implies that $h_{C,t}(v) = h_{C',t'}(v)$ and therefore $b_{C,t}(v) = b_{C',t'}(v)$. We can conclude that, for each vertex v of τ_d and each state s of M , there is only one number $n(v, s)$ such that $b_{C,t}(v) = n(v, s)$ for all critical trees t with 1-leaf v and all accepting computations which visit v in the state s . In other terms, there exists an m -labeling M of the leaves of τ_d such that, for each critical t and each accepting computation C on t the numbering $b_{C,t}$ is compatible with M . This contradicts Lemma 5, as desired. Therefore, the claim is proved.

Let d, t, t', C, C' and v be as given by the above claim. We complete the proof by constructing a tree $t_0 \in T_3^0$ which is accepted by M . Let $z = h_{C,t}(v)$ and $z' = h_{C',t'}(v)$. Let j be minimal such that $z_j \neq z'_j$. We can assume w.l.o.g. that $z_j = 0$ and $z'_j = 1$. Let, for each $i \in \{1, \dots, d\}$, v_i be defined as above, w_i be the 1-sibling of v_i in t and w'_i be the 1-sibling of v_i in t' . We construct t_0 as follows: (i) for each $i < j$, if $z_i = 1$ (i.e., w_i is visited before v in C) then we copy the subtrees rooted at the siblings of v_i from t ; (ii) for each $i < j$, if $z_i = 0$ (i.e., w'_i is visited before v in C') then we copy the subtrees rooted at the siblings of v_i from t' (iii) at the siblings of v_j we root 0-subtrees (this assures that t_0 is a 0-tree); and, (iv) in the subtree rooted at v_j all leaves are labeled 1.

Let $C = c_0, \dots, c_m$, $C' = c'_0, \dots, c'_n$ and let k and k' be such that $c_k = c'_{k'}$ are the configurations in which v is visited.

It is straightforward to check that

- $c_0, \dots, c_k$ is a valid subcomputation on t_0 because all 1-leaves of t that are visited in $c_0, \dots, c_k$ are also 1-leaves in t_0 ;
- $c'_{k'}, \dots, c'_n$ is a valid subcomputation on t_0 because all 1-leaves of t' that are visited in $c'_{k'}, \dots, c'_n$ are also 1-leaves in t_0 ; hence
- $c_0, \dots, c_k = c'_{k'}, \dots, c'_n$ is an accepting computation on t_0 , the desired contradiction.

This concludes the proof of statement (a).

To prove (b) we use a slightly different set U_3^1 of trees. These trees have an additional label, $+$. Inner vertices that are labeled with $+$ have 3 children which are interpreted as threshold gates. Inner vertices that are not labeled with $+$ have only 1 child. Hence, at a $+$ -vertex there are starting 3 paths which lead either to another $+$ -vertex or to a leaf. Now a $+$ -vertex is evaluated to 1, if at least 2 of its 3 descendants ($+$ -vertex or leaf) evaluate to 1. Intuitively, U_3^1 is the same as T_3^1 but the edges of trees in T_3^1 are replaced by paths in U_3^1 . In fact, the proof of the fact that no r -restricted TWA recognizes U_3^1 is almost word for word the proof given in (a) but in the trees that are used, each edge has to be replaced by a path of length $r + 1$. The old vertices are labeled with $+$, the new ones not.

Acknowledgements

We thank Joost Engelfriet, Martin Grohe, Hendrik Jan Hoogetboom, and Clemens Lautemann.

References

1. S. Abiteboul, P. Buneman, and D. Suciu. *Data on the Web : From Relations to Semistructured Data and XML*. Morgan Kaufmann, 1999.
2. A. V. Aho and J. D. Ullman. Translations on a context-free grammar. *Inform. and Control*, 19:439–475, 1971.
3. Y. Bargury and J. A. Makowsky. The expressive power of transitive closure logic and 2-way multi-head automata. In E. Borger, G. Jäger, H. K. Büning, and M. M. Richter editors. *Computer Science Logic*, volume 626 of *Lecture Notes in Computer Science*, pages 1–14. Springer-Verlag, 1991.
4. G. Bex, S. Maneth, and F. Neven. A formal model for an expressive fragment of XSLT. Submitted.
5. R. Bloem and J. Engelfriet. A comparison of tree transductions defined by monadic second order logic and by attribute grammars. To appear in the *Journal of Computer and System Sciences*.
6. A. Brüggeman-Klein, S. Hermann, and D. Wood. Context, caterpillars, tree automata, and tree pattern matching. In *Proceedings of the 4th International Conference on Developments in Language Theory*, 1999.
7. J. Clark. XSL Transformations version 1.0. <http://www.w3.org/TR/xslt>.
8. J. Engelfriet and H. J. Hoogetboom. Tree-walking pebble automata. In J. Karhumäki, H. Maurer, G. Paun, and G. Rozenberg, editors, *Jewels are forever, contributions to Theoretical Computer Science in honor of Arto Salomaa*, pages 72–83. Springer-Verlag, 1999.
9. J. Engelfriet and H. J. Hoogetboom. Private communication.
10. J. Engelfriet, H.J. Hoogetboom, and J.-P. van Best. Trips on trees. *Acta Cybernetica*, 14:51–64, 1999.
11. Z. Fülöp and S. Maneth. Domains of partial attributed tree transducers. *Information Processing Letters*, 175–180, 2000.
12. H. Gaifman. On local and nonlocal properties. In J. Stern, editor, *Logic Colloquium '81*, pages 105–135. North Holland, 1982.
13. F. Gécseg and M. Steinby. Tree languages. In [19], chapter 1.

14. T. Kamimura and G. Slutzki. Parallel and two-way automata on directed ordered acyclic graphs. *Information and Control*, 49(1):10–51, April 1981.
15. T. Milo, D. Suciu, and V. Vianu. Type checking for XML transformers. Proceedings of the *19th ACM Symposium on Principles of Database Systems*, 2000.
16. F. Neven. *Design and Analysis of Query Languages for Structured Documents — A Formal and Logical Approach*. Doctor's thesis, Limburgs Universitair Centrum, 1999.
17. F. Neven and T. Schwentick. Expressive and efficient pattern languages for tree-structured data. Proceedings of the *19th ACM Symposium on Principles of Database Systems*, 2000.
18. A. Potthoff. Logische Klassifizierung regulärer Baumsprachen. Doctor's thesis, Institut für Informatik u. Prakt. Math., Universität Kiel, 1994.
19. G. Rozenberg and A. Salomaa, editors. *Handbook of Formal Languages*. Springer, 1997.
20. W. Thomas. Languages, automata, and logic. In [19], chapter 7.